Democratizing Reliable Knowledge-Seeking with MyGPT: A Privacy-First, Open-Source Retrieval-Augmented Generation Platform

Jaimin Patel[1,2,†,*], Jude Downing[1,2,†], Duccio Malinverni[1,2], Hugh Ferguson[1,2], Steve Mathew D. A.[1,2,4], Inês Chen[2], Madison Sluter[1,2], Balint Meszaros[1,2], Vikas Trivedi[1,2], Shahinur Alam[3], Anthony Woodard[3], Dan Alford[3], and M. Madan Babu[1,2,*]

[1]Center of Excellence for Data-Driven Discovery, Department of Structural Biology, St. Jude Children's Research Hospital, Memphis, TN, USA
[2]Department of Structural Biology, St. Jude Children's Research Hospital, Memphis, TN, USA
[3]School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore, India
[3] Research Information Services, St. Jude Children's Research Hospital, Memphis, TN, USA.
[†]J.P. and J.D. contributed equally to designing the initial prototype for MyGPT.
** Corresponding authors:* jaimin.patel@stjude.org, madan.babu@stjude.org

ABSTRACT
Large language models (LLMs) have transformed knowledge discovery, but their use in sensitive domains is limited by privacy concerns and risks of data leakage. Furthermore, LLMs may hallucinate when queried about information outside their training data, undermining reliability for scientific, clinical, and policy-related tasks. Here, we present MyGPT, a fully open-source, privacy-preserving retrieval-augmented-generation (RAG) platform that uses the latest open-source LLMs and operates locally or within secure institutional environments. MyGPT empowers users to query user-defined libraries such as biomedical literature, confidential clinical documents, and multi-lingual global health policies, while ensuring that sensitive data never leaves the secure environment. We introduce new confidence metrics, including "relevance scores" and "hallucination index," and provide explicit source citation from the document library to help users assess answer trustworthiness. MyGPT represents a reliable open-source alternative to commercial LLM services and democratizes LLM-based knowledge seeking across multiple domains.

KEYWORDS
Large Language Models (LLMs), Generative AI, Generative Pre-trained Transformer (GPT), Artificial Intelligence, Retrieval-Augmented Generation (RAG), Hallucination Index.

INTRODUCTION

Information seeking is fundamental to modern human activity (**Extended Data Fig. 1**). With the rise of Large Language Models (LLMs), users can now ask detailed questions that require synthesis, reasoning, and examination of multiple sources. However, LLMs have limitations that make them unreliable in areas requiring certainty, transparency, and reproducibility, such as scientific research[2,3]. They can generate plausible but incorrect responses, known as hallucination[2], due to insufficient human feedback, biases in the training dataset[4], and the exclusion of private and copyrighted material from training data. LLMs are also limited to data available up to their training cut-off date (e.g., September 2023 for GPT-o1). Moreover, they may not perform well at tasks requiring synthesis of complex concepts for real-world, research-related purposes[2,5] such as literature review, where LLMs may provide biased answers or hallucinated sources, undermining their reliability and transparency[6].

Use of commercial services such as ChatGPT and Gemini raises several important issues. First, they collect user data, queries, and documents to train their AI/ML resources, posing a privacy concern[7]. While such services advise against sharing private, confidential, or sensitive information[5,8], users in academic, clinical, and public institutions can inadvertently leak sensitive data[9]. Consequently, institutions either restrict access or opt for costly enterprise versions[7]. Importantly, commercial services replace their older LLMs with newer ones, hindering reproducible research[9] (e.g., the GPT-o1-mini scheduled to retire on October 2025). Additionally, institutes or scientists working in remote locations may be unable to use commercial services without reliable internet and cloud connections, further complicating access and security concerns[9].

LLM-powered products designed with a focus on privacy and customization may overcome these limitations[1,3,9]. One approach is the retrieval-augmented generation (RAG) concept, where LLMs trained on limited data are aided by additional information retrieved from indexed documents to generate a response[10]. In simple terms, when the user asks a question, LLMs receive it with a set of instructions known as a "prompt". When additional supporting information is provided as "context" along with the "prompt", improved responses and reduced hallucinations have been observed[2]. Such "context" can be retrieved from user-defined libraries of documents, ensuring they remain private and increasing the response accuracy.

Here, we present MyGPT, an open-access, locally run, and customizable RAG pipeline that enables researchers to ask questions from a curated library of documents or publications (**Fig. 1a-b**). MyGPT presents the exact sources of its responses to increase transparency and accuracy; provides new confidence metrics to identify likely hallucination; and features a user-friendly interface for customizing various components within the pipeline. MyGPT's modularity provides an easy and future-proof way of swapping components with newly released ones, such as domain-specific LLMs and embedding models[9]. Thus, MyGPT democratizes reliable knowledge seeking by filling a gap in the current arsenal of knowledge-seeking tools (**Extended Data Fig. 1**).

RESULTS

Outline of the MyGPT architecture and the workflow

MyGPT architecture is composed of three sections: a user interface (UI), a backend server, and an LLM server (**Fig. 2a**). The UI and backend server function to pre-process the library of documents and perform real-time question-and-answer (Q&A), while the LLM server is key for response generation.

In the pre-processing step, the UI first receives a library of documents, either via a reference management tool (e.g., Zotero[11]; **Methods**) or direct upload. The backend server divides the documents into smaller chunks. Chunking is essential for effective information processing, since LLMs have a limited context window size and currently cannot accept entire or large documents. The chunks are then vectorized, i.e.

converted into a vector of numbers that capture similarity and thus acquire specific meanings, as defined by the embedding model. Different embedding models, trained on domain-specific data, can enhance the response of an LLM by capturing specific relationships between words and sentences in that particular domain[12]. The vectorized representations of knowledge sources are then stored in a searchable database hosted through the backend (**Fig. 2b,** blue arrows). The embedding models organize related terms close to each other in the vector space[12]. This concept also applies to sentences and paragraphs, where entities with similar meanings are positioned closer to one another, showing better performance in similarity searches[13], as illustrated in **Extended Data Fig. 2**.

After pre-processing, the UI displays the library as a list of documents and a chat box for real-time Q&A (**Fig. 1b**). Users can upload additional documents and customize the various components of MyGPT. When a user asks a question, it is sent to the backend server to be converted into a vector embedding by the same embedding model used to preprocess the library and passed to the vector database (**Fig. 2b,** red arrows). The vector database's similarity search algorithm compares the question to the chunks from the library, to identify similar chunks that are then provided to the LLM server as a "context" for the query, enhancing the LLM response[14]. The LLM, which runs locally on a personal computer or a secure server/cloud infrastructure, performs context-based Q&A, generates a response, and sends it back to the UI (**Fig. 2b,** green arrows). The UI then displays the source material used as context to generate the response with their exact location (e.g., document pages with highlighted sentences or paragraphs) (**Fig. 1b**). In addition, the UI displays confidence metrics calculated by the backend server that can help identify instances of hallucinations (**Fig. 1b;** see below and **Methods**).

MyGPT customization options

The different sections within MyGPT are composed of modularized components (**Fig. 2b**) that can be replaced or customized through the UI (**Extended Data Fig. 3**), as summarized below:

- *Chunking:* MyGPT offers two chunking methods: fixed-size or document-structure-preserving recursive chunking (**Methods**). Both allow users to adjust chunk overlap (i.e., the number of characters shared between two consecutive chunks[15,16]) and, later, to focus on specific section within a document or standard sections across all documents.

- *Embedding models:* MyGPT users can deploy any open-source embedding model available from Hugging Face (a platform to host and share open-source AI/ML resources) or via Ollama (an LLM hosting tool). Over 400 different[17,18] embedding models are available, including general-purpose embedding models, such as nomic-embed-text, or a domain-specific embedding models, such as MedCPT.

- *Vector space distance function* (i.e., similarity search metric): To identify relevant chunks, MyGPT users can choose from three distance functions: Euclidian distance, cosine similarity, and Inner product (**Methods**).

- *Large language models (LLMs):* MyGPT provides a UI for downloading several popular open-source LLMs via Ollama, or any of the additional 100+ open-source LLMs available there using a command-line interface, enabling their local deployment and circumventing the need to connect to proprietary online services[9,19]. Ollama's performance depends on the available GPU, making it an essential component for running MyGPT. We performed comprehensive benchmarking to evaluate several LLMs in terms of ideal hosting environment, minimum requirements, average speed of response to a user query, and cost estimates of hosting in a cloud infrastructure (**Extended Data Tables 1–3)**. For example, relatively small Llama3:8b and Llama2:7b[20] (~8 billion and ~7 billion parameters, respectively) can run on a machine with a single GPU and generate answers quickly. In contrast, a large LLM such as Llama3:70b (~70 billion) must be hosted on a machine with a powerful GPU for optimal performance but can generate detailed answers. Users can also interact directly with the LLMs by turning off the RAG pipeline through the UI.

- *Creativity vs precision and prompting:* LLM models generate text by predicting the next words in the sequence, and this can be customized by adjusting specific LLM parameters: top-K (the most probable words to consider for the prediction), top-P (the threshold of probabilities of words), and temperature (the randomness of words selected by top-P and top-K). In this way, users can control the creativity or precision of the responses generated by the LLM. Users can also limit the output from the LLM by changing the default prompt through the UI, e.g. answer in a single word (yes/no), to restrict creativity.

- *Model context protocol (MCP)*: This open-standard developed by Anthropic[21] facilitates integration of LLM applications with external public or private tools and databases. MyGPT offers a basic implementation of the MCP server and client, enabling its integration with other private LLM applications and data sources hosted within the same AI ecosystem (**Supplementary Information)**. This enables advanced users to utilize already established MCP servers with public or private tools with MyGPT (example, using Protein Data Bank MCP tools with MyGPT, **Supplementary Information**), or MyGPT as an essential and robust RAG component in private agentic workflows.

Confidence metrics

In RAG applications, the risk of hallucinations increases if relevant information is missing from the documents, therefore users must evaluate the answers provided by the LLM and verify their sources to ensure accuracy. To address this challenge, in addition to presenting the specific source from the library used to generate the response, we also developed and implemented "relevance scores" both for user queries and MyGPT-generated answers, as well as a "hallucination index". MyGPT provides built-in functionality to calculate these metrics, which can be further optimized for specific use cases. We define these metrics below (see also **Methods**):

- *Question Relevance Score (QRS)*: provided when the user submits the question, it serves as a metric to assess whether the user-provided library may have the necessary information to respond correctly to the question.

- *Answer Relevance Score (ARS):* it measures the extent to which MyGPT used the context (i.e., source material) retrieved from the documents to generate the answer.

- *Hallucination Index (HI):* a weighted linear combination of QRS and ARS, this score helps users identify likely instances of hallucination and serves as a confidence metric for the accuracy and reliability of the generated answers. In this way, MyGPT can flag likely instances of hallucinations, either because the question is less relevant to the library or because the context retrieved from the document lacks sufficient information for the LLM to generate an accurate response.

LLM hallucination can be independently evaluated by other methods such as SelfCheckGPT[22] and semantic entropy[23] through generation of multiple responses, but they are not well-suited for RAG contexts. This is because, SelfCheckGPT and semantic entropy require substantial GPU resources, and SelfCheckGPT is ineffective for smaller LLMs such as GPT-3 and Llama[22]. Furthermore, if the correct context is provided, even smaller LLMs can produce accurate answers. Thus, by providing QRS, ARS and HI metrics, and explicit source citing, MyGPT offers a comprehensive and generalizable method to quantify the correctness of LLM-generated answers, thereby increases the reliability and transparency of RAG applications. To ensure that the implemented RAG concept works as expected, we have benchmarked and demonstrated the usability of MyGPT and the confidence metrics using two different types of document libraries and question and answer benchmark datasets: a publicly available library of biomedical articles (**Supplementary Information**) and an entirely private library containing just this document (i.e., the MyGPT manuscript).

Benchmarking MyGPT for queries on private documents

We assessed the usability of MyGPT for private documents. There is growing concern and evidence that LLMs are trained on public data, including research articles[24], making it challenging to determine which specific publications were included in the training datasets for the various LLMs. To circumvent this concern, we used the MyGPT manuscript (which could not have been included in the LLM training dataset) as a private document, allowing us to test how the RAG answers questions regarding a private document.

We curated a dataset of 40 questions related to the MyGPT manuscript from 2 domain experts: 25 were related to information present in the manuscript (relevant questions), and 15 were related to the topic but the manuscript lacked the information to answer them (irrelevant questions) (see also **Methods**). MyGPT answered all 25 relevant questions correctly and failed to answer the 15 irrelevant questions (**Fig. 3a-c**). In contrast, the LLM alone generated incorrect—or hallucinated—answers for all 40 questions. The confidence metrics (ARS, QRS, and HI) obtained using MyGPT could differentiate between correct and incorrect answers and identify instances of hallucination decisively (**Fig. 3d-f**). We observed that QRS ($p = 1\text{x}10^{-3}$) and ARS ($p = 1.9\text{x}10^{-2}$) were significantly higher for relevant questions and correct answers, and HI ($p = 1.7\text{x}10^{-3}$) were significantly higher for irrelevant questions and incorrect answers. These findings demonstrate that MyGPT (and the confidence metrics) works as expected and outperforms standalone LLMs when using private documents.

Use cases for MyGPT

In the following section, we describe a few common use cases for MyGPT in the domains of basic sciences, clinical sciences, and global health as well as discuss our implementation efforts.

Biomedical literature: MyGPT can be useful to query biomedical literature, particularly when exploring new literature (not included in LLM training) or when querying several hundred articles collected by research groups. MyGPT's confidence metrics and source surfacing capability enables researchers to assess the trustworthiness and accuracy of LLM-generated answers, while keeping the knowledge base constrained to a selected library of literature. MyGPT's ability to contextualize multiple documents enables researchers to ask complex questions that can only be answered by gathering related information from various articles, analyzing them, and generating a final answer that benefits from the reasoning capabilities of several open-source LLMs, such as gpt-oss (by OpenAI). To illustrate this use case, we created a library of GPCR articles (**Methods**); when asked a complex question (provided by a domain expert), MyGPT generated an answer with high relevance scores and low hallucination index (**Figure 4**).

Private clinical documents: For use cases that require privacy and confidentiality, MyGPT can be securely deployed within a hosting environment approved and maintained by institutions such as hospitals and clinical settings. As a demonstration, we implemented MyGPT for use by the Department of Bone Marrow Transplantation and Cellular Therapy (BMTCT) at St. Jude (**Methods**), enabling staff to easily access and query Standard Operating Procedures (SOP) documents. Doctors and nurses can use MyGPT to query specific information from a library of SOPs (**Extended Data Fig. 4**) and verify it against the original documents. Since these SOPs are meticulously prepared and approved through a rigorous process at each hospital, responses generated by the language model should be accurate, institution-specific, and verifiable with source citations. MyGPT may also be utilized for other confidential clinical documents, such as patient reports and Electronic Health Records (EHRs).

Multi-lingual global health policies: MyGPT allows users to interact with documents in various languages without concerns about language compatibility. This is possible due to its modular design, which allows users to easily switch between different embedding models, including those trained on multiple languages, such as bge-m3, Paraphrase-Multilingual, and Snowflake-Arctic-Embed2. When a user uploads a document in a language other than English, the embedding model can categorize the content alongside other chunks with similar meanings, irrespective of the original language. Although a user may not be familiar with the context in an unfamiliar language, the language model used for generating the final answer can recognize and translate it to the language in which the user asks their question. We leveraged this capability to implement MyGPT for the department of Global Pediatric Medicine, which manages health policy documents from over 65 countries, encompassing more than 150 hospitals and five different languages (**Extended Data Fig. 5**).

We anticipate, MyGPT can be utilized in various environments, including routine tasks in hospitals, human resources, administration, legal fields, and commercial contexts, as well as for students and public use (**Figure 5**).

Considerations and limitations

MyGPT uses the textual content from documents to create a vector database, and it is currently unable to interpret tabular data, figures, mathematical equations, formulas, or chemical structures. This limitation can be addressed in the future with multimodal (text + images) LLMs. Additionally, the user needs to provide the documents to create the library, as MyGPT does not automatically retrieve all relevant publications on a topic of interest; if supporting or contradicting information is absent from the user-provided library and/or was not included in the LLM training data, MyGPT could generate a biased response by citing only the available source in the library.

While ARS, QRS, and HI metrics can help identify instances of hallucination, they do not eliminate them and may need to be optimized to distinguish between correct and incorrect responses for specific libraries and scientific domains. These metrics rely on the properties of user input, such as question type and clarity and on MyGPT parameters such as the embedding model used. In addition, users need to install MyGPT on an individual device for personal use, and on Edge AI devices, a server, or a secure cloud for institutional use, requiring maintenance as an internal web resource. The size of the library can influence the number of chunks per library and impact the uploading time, but it does not affect the duration of different steps in real-time Q&A once uploading is complete (**Extended Data Table 4)**. In our ongoing efforts, we also observed that advanced semantic chunking, which is computationally expensive, does not provide a substantial performance improvement[25], but this needs further studies. Despite these considerations, local installation capabilities allow users to customize the pipeline according to their requirements, securely deploy it within a firewall, eliminate the need to send confidential information to third parties, minimize data leakage, and make MyGPT more affordable than commercial cloud-hosted LLM products for individuals, research groups, and organizations of all sizes.

DISCUSSION

MyGPT fills a crucial gap in knowledge discovery tools by enabling users to extract insights from multiple private documents efficiently. It also allows researchers to search their private and curated libraries containing the most recent scientific literature on a topic and obtain reliable answers from several publications simultaneously. Importantly, MyGPT is open-source, easy to install, portable, and customizable, making it widely accessible as new LLMs and embedding models emerge. To identify the optimal combination of MyGPT components (LLMs, embedding model, distance function), we suggest selecting top-performing LLMs from leaderboards such as Chatbot Arena[26,27] and embedding models from the Massive Text Embedding Benchmark (MTEB) Leaderboard[17,18] recommended by sentence transformer studies[28]. For other customizations, domain-specific LLMs or embedding models beyond the leaderboards, users can also utilize automated evaluation frameworks such as Retrieval-Augmented Generation Assessment (Ragas)[29,30] with MyGPT APIs.

We note that MyGPT includes solutions akin to those offered by commercial products such as ChatPDF, NotebookLM, PrivateGPT, and open-source products such as Verba, LlamaIndex, and BioChatter (**Extended Data Tables 5)**. However, in contrast with those products, MyGPT is a complete, end-to-end pipeline built with open-source tools that are small enough for local installation on a laptop/personal computer, Edge AI devices or institutional servers/VM, or cloud. MyGPT can be installed and initiated by a script orchestrating prebuilt Docker images, eliminating complex debugging and environment dependency for less tech-savvy audiences.

MyGPT offers essential features while providing users with complete control over their data. In research and industry, a substantial amount of knowledge is locked in private documents, necessitating a solution for a contextual, retrievable, and scalable knowledge base. Future RAG development with agentic architecture must access this private knowledge based on user permissions. Agentic workflows can effectively integrate MyGPT's confidence metrics to filter out low-relevance references and rephrase less relevant questions, thereby enhancing decision-making and improving accuracy (e.g., context engineering). Furthermore, the various parameters collected during each Q&A session, along with the types of questions for different domains, can be used to train a low-latency machine learning model or neural network to advance confidence metrics further. Current commercial Deep Research applications do not support private documents or need a subscription to commercial services. In contrast, MyGPT offers an open-source API with built-in user access control, enhancing agentic workflows, and the Model Context Protocol (MCP) to enhance ongoing efforts such as Open Deep Research[31] and multi-agent systems for healthcare (MASH)[32].

MyGPT is also valuable in non-academic settings, such as hospitals, private sector and legal organizations (**Figure 5**). With the ability to run on Edge AI devices and handle documents offline (**Extended Data Fig. 6**), MyGPT serves as a cost-effective, portable alternative to commercial LLM services, democratizing access to reliable knowledge-seeking tasks. Since vector embeddings for specific libraries are independent of GPT models, sharing pre-made MyGPT libraries with the community can accelerate and broaden opportunities for knowledge seeking and scientific discovery. This empowers researchers to utilize publications from open research initiatives, such as PubMed Central[33] or openRxiv[34], to create private and open-source instances of AI-powered generalized literature research tools such as Consensus[35]. It can also facilitate the accelerated development of agentic tools, similar to CRISPR-GPT[36] and PrimeGen[37], by providing a robust RAG component. As we look to the future of generative AI, personalized GPT products are increasingly becoming a reality. As commercial AI companies strive to contextualize personal data, an open-source, privacy-focused retrieval-augmented generation (RAG) framework presents a compelling path for generative AI that balances accessibility, accountability, and user autonomy.

REFERENCES

1 Xu, R., Feng, Y. & Chen, H. ChatGPT vs. Google: A Comparative Study of Search Performance and User Experience. *ArXiv* **abs/2307.01135** (2023).

2 Cascella, M., Montomoli, J., Bellini, V. & Bignami, E. Evaluating the Feasibility of ChatGPT in Healthcare: An Analysis of Multiple Clinical and Research Scenarios. *J Med Syst* **47**, 33 (2023). https://doi.org:10.1007/s10916-023-01925-4

3 Eloundou, T., Manning, S., Mishkin, P. & Rock, D. GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models. *ArXiv* **abs/2303.10130** (2023).

4 Alkaissi, H. & McFarlane, S. I. Artificial Hallucinations in ChatGPT: Implications in Scientific Writing. *Cureus* **15**, e35179 (2023). https://doi.org:10.7759/cureus.35179

5 OpenAI. *How does OpenAI use my personal data?*, https://openai.com/policies/row-privacy-policy/ (2024).

6 Kwong, J. C. C., Wang, S. C. Y., Nickel, G. C., Cacciamani, G. E. & Kvedar, J. C. The long but necessary road to responsible use of large language models in healthcare research. *NPJ Digit Med* **7**, 177 (2024). https://doi.org:10.1038/s41746-024-01180-y
7 Lin, Z. Techniques for supercharging academic writing with generative AI. *Nat Biomed Eng* **9**, 426-431 (2025). https://doi.org:10.1038/s41551-024-01185-8
8 Gemini. *Gemini Apps Privacy Notice*, https://support.google.com/gemini/answer/13594961?visit_id=638586578410247935-92666435&p=privacy_notice&rd=1#privacy_notice (2024).
9 Hutson, M. Forget ChatGPT: why researchers now run small AIs on their laptops. *Nature, Technology Feature* (2024).
10 Lewis, P. *et al.* Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *ArXiv* **abs/2005.11401** (2020).
11 Zotero.org. https://www.zotero.org/
12 AWS. *What are embeddings in machine learning?*, https://aws.amazon.com/what-is/embeddings-in-machine-learning/
13 Jing, Z. *et al.* When Large Language Models Meet Vector Databases: A Survey. *arXiv preprint arXiv:2402.01763* (2024).
14 Omar, M., Ullanat, V., Loda, M., Marchionni, L. & Umeton, R. ChatGPT for digital pathology research. *The Lancet Digital Health* **6**, e595-e600 (2024). https://doi.org:10.1016/S2589-7500(24)00114-6
15 Peter, A. *What Chunk Size and Chunk Overlap Should You Use?*, https://dev.to/peterabel/what-chunk-size-and-chunk-overlap-should-you-use-4338 (2023).
16 Hoang, L. *How Chunk Sizes Affect Semantic Retrieval Results*, <https://ai.plainenglish.io/investigating-chunk-size-on-semantic-results-b465867d8ca1> (
17 Muennighoff, N., Tazi, N., Magne, L. & Reimers, N. MTEB: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316* (2022).
18 *Massive Text Embedding Benchmark (MTEB) Leaderboard*, https://huggingface.co/spaces/mteb/leaderboard
19 Jeffery Morgan, M. C. *Ollama*, https://ollama.com/ (2021).
20 Touvron, H. *et al.* Llama 2: Open Foundation and Fine-Tuned Chat Models. *ArXiv* **abs/2307.09288** (2023).
21 Model Context Protocol (MCP) (GitHub https://github.com/modelcontextprotocol, 2024).
22 Manakul, P., Liusie, A. & Gales, M. J. F. SELFCHECKGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *2023 Conference on Empirical Methods in Natural Language Processing (Emnlp 2023)*, 9004-9017 (2023).
23 Farquhar, S., Kossen, J., Kuhn, L. & Gal, Y. Detecting hallucinations in large language models using semantic entropy. *Nature* **630** (2024). https://doi.org:10.1038/s41586-024-07421-0
24 Gibney, E. Has your paper been used to train an AI model? Almost certainly. *Nature* **632**, 715-716 (2024). https://doi.org:/10.1038/d41586-024-02599-9
25 Qu, R., Tu, R. & Bao, F. S. 2155-2177 (Association for Computational Linguistics).
26 *LMSYS Chatbot Arena Leaderboard*, https://lmarena.ai/?leaderboard
27 Chiang, W.-L. *et al.* Chatbot arena: An open platform for evaluating llms by human preference. *arXiv preprint arXiv:2403.04132* (2024).
28 *Pretrained Models* <https://www.sbert.net/docs/sentence_transformer/pretrained_models.html> (
29 *Ragas (RAG ASsement)* https://docs.ragas.io/en/stable/index.html
30 Es, S., James, J., Espinosa-Anke, L. & Schockaert, S. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217* (2023).
31 Open Deep Research (GitHub https://github.com/langchain-ai/open_deep_research).
32 Moritz, M., Topol, E. & Rajpurkar, P. Coordinated AI agents for advancing healthcare. *Nat Biomed Eng* **9**, 432-438 (2025). https://doi.org:10.1038/s41551-025-01363-2
33 Central, P. *PubMed Central*, https://pmc.ncbi.nlm.nih.gov/ (2000).
34 OpenRxiv. *OpenRxiv*, https://openrxiv.org/ (2025).
35 consensus. *Consensus - AI-powered search engine for scientific research papers*, https://consensus.app/home/about-us/ (2021).
36 Qu, Y. *et al.* CRISPR-GPT for agentic automation of gene-editing experiments. *Nat Biomed Eng* (2025). https://doi.org:10.1038/s41551-025-01463-z
37 Wang, Y. *et al.* Accelerating primer design for amplicon sequencing using large language model-powered agents. *Nat Biomed Eng* (2025). https://doi.org:10.1038/s41551-025-01455-z

38 Trychroma.com. https://www.trychroma.com/
39 Lopez, P. GROBID: Combining Automatic Bibliographic Data Recognition and Term Extraction for Scholarship Publications. *Lect Notes Comput Sc* **5714**, 473-474 (2009).

FIGURES

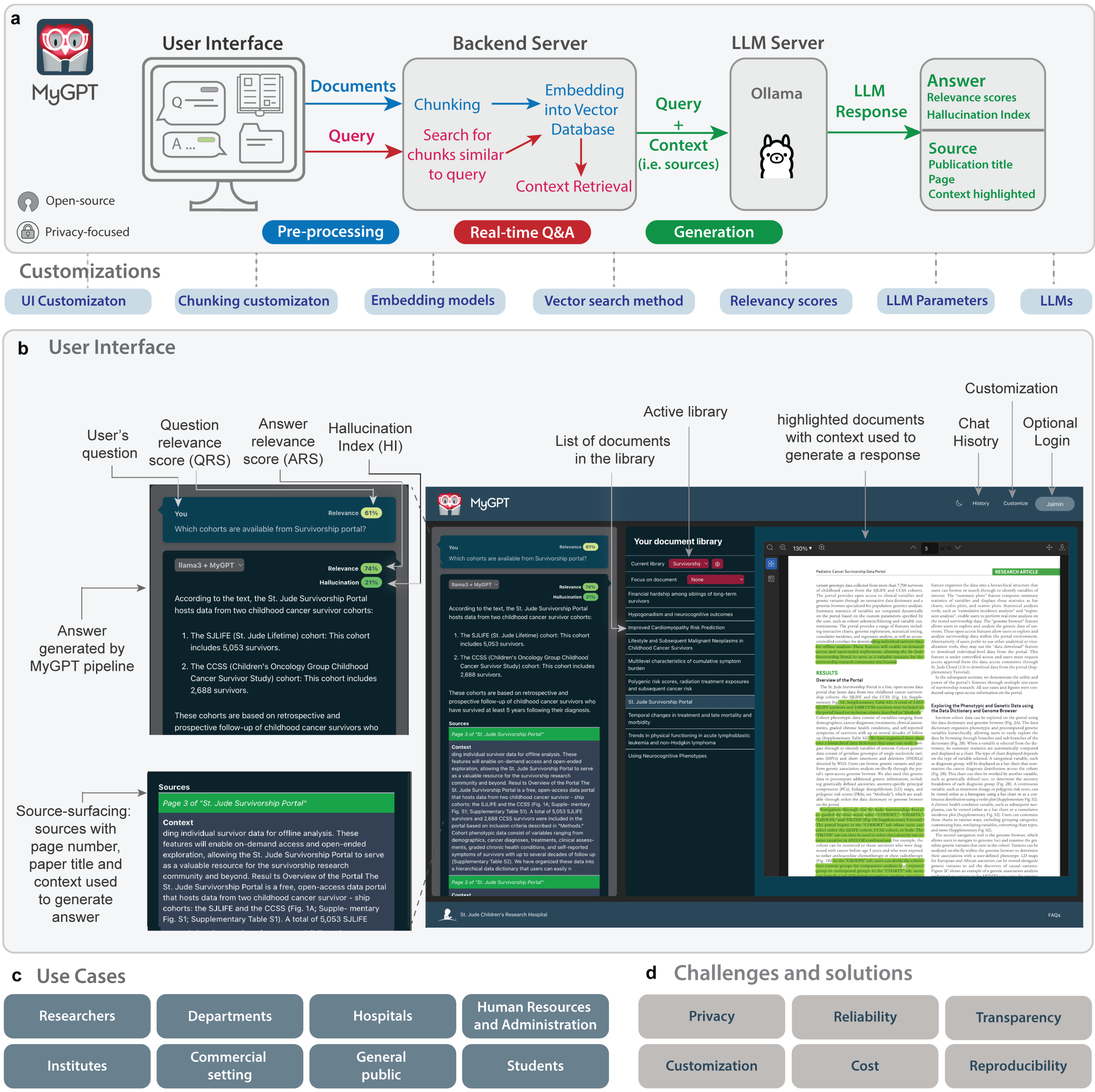


Figure 1. MyGPT pipeline and user interface. a, MyGPT is comprised of three components: front end, back end, and LLM server, which facilitate three main steps: document library pre-processing, real-time Q&A, and answer generation with confidence matrices and source citation. **b,** MyGPT user interface is comprised of 3 panels. The left-most panel is used for real-time Q&A and displays user query with question relevance score (QRS), LLM generated answer with answer relevance score (ARS), and hallucination index (HI). The answer is also accompanied by sources, including the document and paragraphs (with exact page numbers) used to generate the answers. Clicking on the source activates the document from the middle panel, which displays all the documents from the library, while the right-most panel shows the document page and the highlighted context.

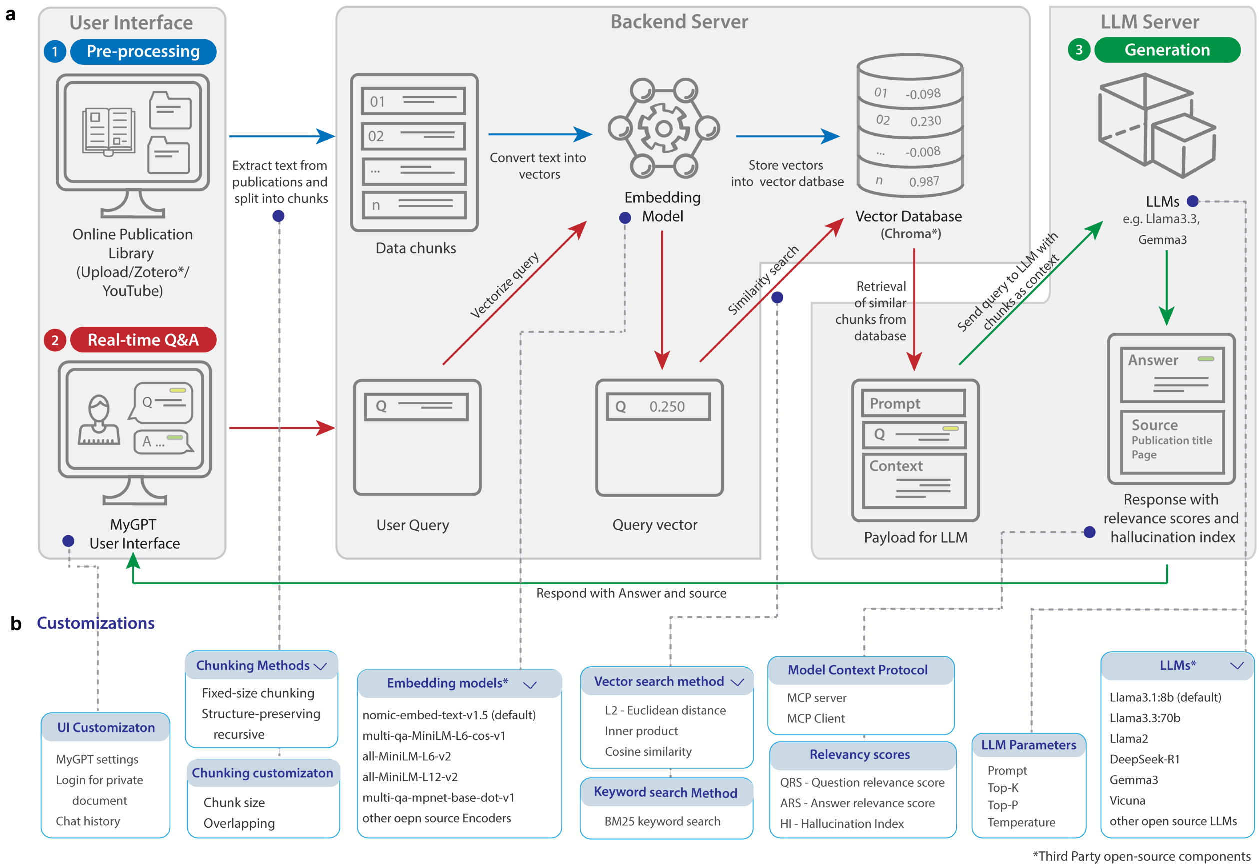


Figure 2. An overview of the MyGPT pipeline and customizations. a, In-depth view of each component within the MyGPT pipeline. Blue arrows show the process followed by user-uploaded documents. First, a user uploads documents, then those are pre-processed and converted from text into chunks and stored in a vector database. Red arrows show the process followed by user queries. The real-time Q&A converts user questions or "queries" into vectors and then finds the most similar chunks from the vector database. The closest chunks are then supplied to the LLM as context for the user query to produce a response (green arrows); The pipeline provides the user with an explicit context source from the library with the generated answer and relevancy scores for question (QRS), answer (ARS), and hallucination index (HI) to identify likely instances of hallucination in the generated response. **b,** MyGPT Customizations. Users can modify the MyGPT pipeline through the user interface at various stages, such as chunking method, chunking customizations, embedding models, vector search methods, keyword search method, relevancy scores, LLMs, and creativity parameters. MyGPT also provides MCP (model context protocol) capabilities, allowing integration with other LLM applications and access to private data.

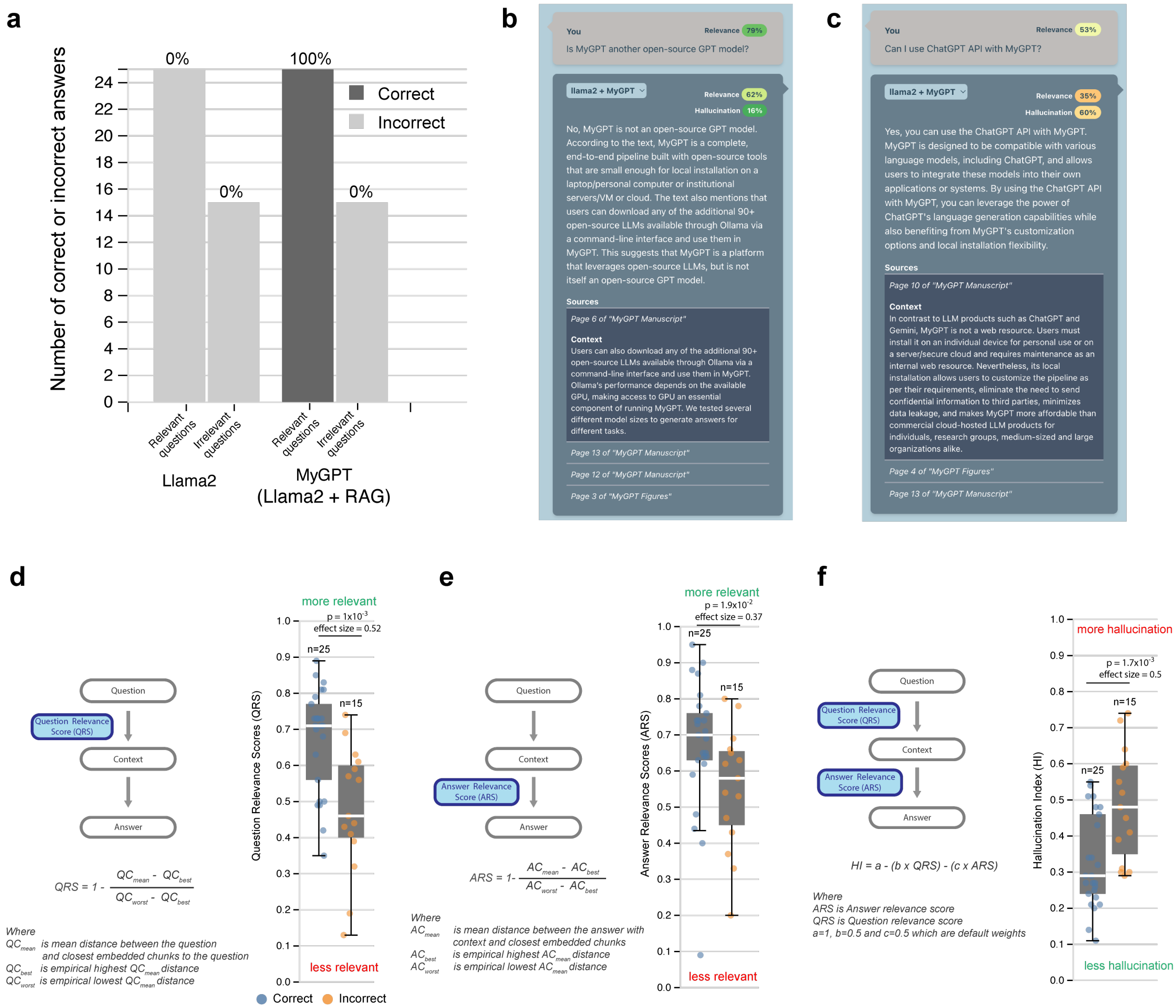


Figure 3. Evaluation of MyGPT (Llama2 + RAG) for a private document. a, Frequency of answer correctness with Llama2:7b alone and MyGPT (Llama2:7b + RAG). Users were asked if the generated answers were correct or not, and they provided feedback with "Yes" (dark grey), indicating the answer generated was correct, and "No" (light grey), indicating the answer generated was incorrect. The 40 questions were designed to have 25 questions where the necessary information to answer was in the private document and 15 irrelevant questions (i.e., questions lacking the necessary information in the document to answer them correctly). **b,** Example of relevant question and correct answer, indicated by high QRS, ARS, and low HI. **c,** Example of irrelevant questions and incorrect answers, indicated by low QRS, ARS, and high HI. **d,** Question relevancy score (QRS) distribution for questions of correct and incorrect answers (p=1x10^{-3}; effect size=0.52; Wilcoxon rank-sum test). **e,** Answer relevancy score (ARS) distribution for correct and incorrect answers (p=1.9x10^{-2}; effect size=0.37; Wilcoxon rank-sum test). **f,** Default Hallucination Index (HI) for correct and incorrect answers using arithmetic means between QRS and ARS (p=1.7x10^{-3}; effect size=0.5; Wilcoxon rank-sum test).

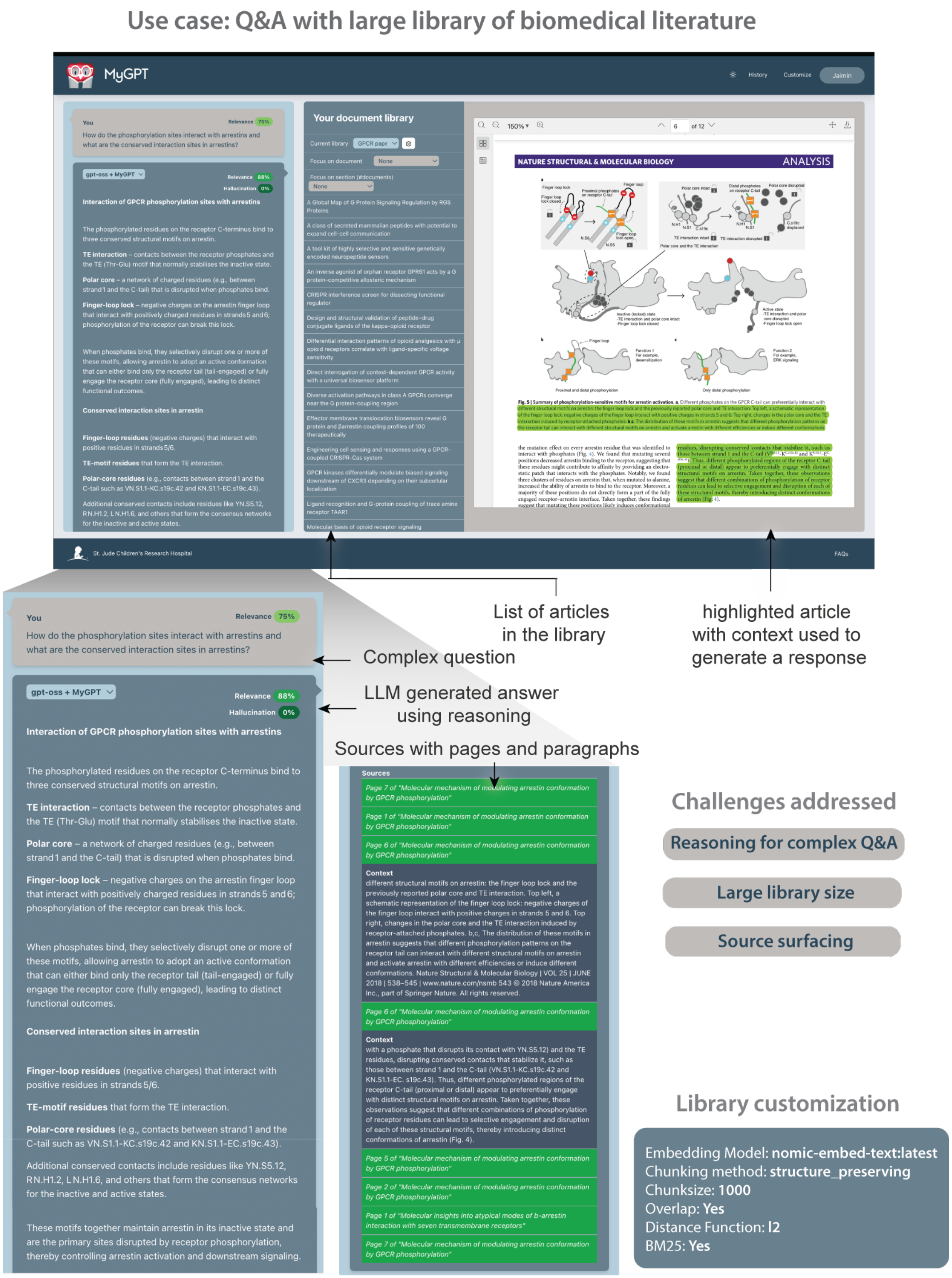


Figure 4. **Use case: Q&A with large library of biomedical literature.** Example of a complex question and answer for the library of Biomedical literature on G protein-coupled receptors (GPCRs). The example shows the user's question and answer, along with the highlighted section of the articles used by LLM (gpt-oss:12b) to generate the answer. This use case addresses the critical challenge of retrieving related information from multiple sections of an article and synthesizing an answer after analyzing the data. The

library was created using the nomic-embed-text:latest embedding model, which utilizes the reasoning capabilities of the LLM (gpt-oss:20b) to generate an answer.

a

Users	Documents	Use cases	Question types
Researchers	Publications Research talks Electronic notebooks	Literature review Talk summary Experiment summary	Summarization Keyword Yes/no Data query Complex question
Departments	Collaboration material Reports	Project on-boarding Reduce domain-specific barrier	Summarization Complex question
Hospital	Electronic Health Records (EHR) Patient care records	Summarize health records Patient on-boarding Personal assistant	Summarization Yes/no Data query
Institute	Internal wiki Project and annual reports	Internal information-seeking Orientation Quicker key insights	Summarization Complex question Data query
Human Resources and Administration	Work Policies Job Posts Candidate Resume	FAQs Institutional chat-bot On-boarding assistant Talent acquisition	Summarization Yes/no Data query
Students	Lectures Reference books Digital textbooks Educational videos	Deeper understanding of subjects Reviewing topics Different material and formats	Summarization Complex question Keywords
Commercial setting	Confidential material Legal documents	Secure and private Q&A Quicker information retrieval Quicker Insights	Yes/no Data query
General public	Personal journals and notebooks	Personal knowledge Exploration and idea generation	Summarization Complex question

b

Attributes	Challenges	MyGPT solutions
Privacy	No privacy[2]	Enhanced privacy, use on devices or secure cloud infrastructure
	Can't use confidential or proprietary documents	Users can use any confidential or proprietary documents
Reliability	No hallucination detection[1]	Identifiable hallucinations with Hallucination index (HI)
	Knowledge limitation until training cut-off date	LLM knowledge aided by user-provided documents
	No confidence metrics or reliability[1]	Confidence metrics to increase reliability and transparency
Transparency	No transparency[1] and source-citing	Enhanced transparency with source-citing
Customization	Limited LLM choices	Employ any existing or upcoming open-source LLMs
	Limited LLM parameters customization	Numerous MyGPT pipeline customizations for optimization
Cost	High cost[2]	Free and open-source
Reproducibility	No reproducibility due to retirement of old LLMs[2]	Enhanced reproducibility by using open-source LLMs

Figure 5. Use cases, and challenges of existing GPT products and solutions provided by MyGPT. **a,** Different use cases for MyGPT for information-seeking tasks from individual users to groups of users from academia, industry, and public. Users have the flexibility to modify various MyGPT parameters based on their use-case. **b,** Existing GPT solutions are prone to data leaks and compromise privacy, or

enterprise GPT versions are only affordable to some institutions. MyGPT addresses these challenges along with several others and provides a software solution that fills the unmet needs of existing GPT solutions.

METHODS

MyGPT architecture

The architecture of the MyGPT pipeline consists of three sections: (i) a user interface (UI), (ii) a backend server, and (iii) an LLM server (**Fig. 2**). The UI and backend serve a dual purpose of library pre-processing and real-time question and answering.

For library pre-processing, the UI receives the documents and customizations for the chunking method, selected embedding model, and distance function and sends them to the backend. The backend then converts documents into chunks and chunks into embeddings using a user-selected embedding model and stores them in a vector database. For different libraries, the backend creates separate sections/collections and stores in the same vector database with necessary metadata about the document type, title of documents, and user. This enables multiple users to use the same library and vector database without the need for access control and data leakage of information between libraries of a single user or multiple users. The UI also provides an interface to manage libraries; users can add, switch, or delete the documents in the libraries. Deleting the library will delete all the documents, vector embeddings from the vector database, and the chat history for that specific library from the backend database.

For real-time question and answering, the UI receives the question and sends it to the backend with a default prompt. The backend server converts the question into an embedding (using the same embedding model as the one used for processing the library) and sends the encoded question to the vector database to retrieve the nearest chunks (i.e., context) to provide to the LLM server for a response. We have incorporated several open-access LLMs into the MyGPT pipeline, including Llama3, Llama2, Mistral, Vicuna, and Gemma, with Llama3 set as the default option (**Supplementary Table 1**). The main purpose of these LLMs is to receive a prompt – a combination of the user's query and context from the vector database – and to respond with an LLM-generated answer using the information provided in the context to generate the response. The UI also provides options to change the default prompt or creativity vs precision settings before the LLM generates the answer. The LLM section of the pipeline can run independently of MyGPT and be used for purposes other than MyGPT (**Fig. 2**). In addition, a single LLM instance can power multiple instances of MyGPT to reduce cost and maintenance.

The MyGPT pipeline uses the following third-party, open-access tools to orchestrate user interaction with LLMs for the three sections mentioned above:

- **Reference management tool (UI)**: We provide the option to create the document library using Zotero[11], a free reference management tool for collecting and annotating publications. Zotero users can create a private or shared library of publications. Alternatively, MyGPT users can upload documents directly without using any reference management tool.
- **Embedding model (Backend)**: We utilize a Python library sentence transformer that converts text, paragraphs, or images into vectors. Users can then search these vectors by finding similar vectors in the vector space. MyGPT uses multi-qa-MiniLM-L6-cos-v1[6] (sentence transformer), an open-access embedding model available as one of the embedding models in the Chroma database, an open-access embedding resource. The user has the option to use other embedding models available via Ollama and Huggingface to be used with MyGPT.
- **Vector database (Backend)**: We use Chroma database[38], an AI-native open-source vector embedding database to store all the chunks from user-provided documents locally as vectors, allowing for easy and fast searching within the vector database.
- **LLM hosting library (LLM server)**: We use Ollama[19], an LLM hosting library that enables users to employ open-source LLMs locally without the need to connect to proprietary online services. MyGPT employs an API architecture to utilize open-source LLMs that are currently available via Ollama or will

be made available in the future. We recommend Llama3:8b for personal laptops and Llama3:70b for Ollama installed on servers/VMs. MyGPT also provides a user interface (UI) to install other LLMs such as Llama2, Gemma, Gemma2, Vicuna, and Mistral. Once installed, the MyGPT UI will display the various LLMs available for use in the pipeline (**Extended Data Fig. 3**).

Documents chunking methods
MyGPT provides two methods for chunking text:

- **Fixed-size chunking method**: MyGPT divides the text from the user-provided documents into chunks of a predetermined number of characters, from start to end. By default, MyGPT uses a chunk size of 1000 characters, but users can choose from options of 500, 750, 1000, or 1200 characters. Additionally, users can adjust the chunk overlap, which refers to the number of characters shared between two consecutive chunks. If this option is selected, MyGPT automatically applies a 20% overlap between adjacent chunks to maintain the meaning of sentences at the end of each chunk.

  Document-structure-preserving recursive chunking: MyGPT first identifies the major hierarchical structures within the documents, including sections, subsections, and other semantic elements. This is achieved by leveraging the table of contents available in the PDF's metadata, or, in the absence of this information, by utilizing the GROBID[39] document parsing library. GROBID's XML outputs provide structured boundaries that preserve the organization of academic papers. Once the sections are identified, each section is treated as an individual entity for subsequent chunking. Unlike fixed-size chunking, this approach first identifies paragraphs within each section. The recursive approach attempts to split the text at subsection boundaries, then paragraph breaks, and finally sentence boundaries. If a paragraph exceeds the chosen chunk size (e.g., 1000 characters), it will be further split without breaking sentences mid-way. This method also allows users to configure overlap for adjacent chunks. By considering the document hierarchy, this approach maintains semantic coherence at the most significant structural levels while adhering to the defined token limits.

Vector embedding of textual chunks and identification of chunks for LLM context
MyGPT converts chunks of text from the library into high-dimensional vectors. MyGPT splits the text from the provided documents with one of the methods mentioned in the previous section. Then MyGPT converts these chunks into vectors using multi-qa-MiniLM-L6-cos-v1 or user-selected embedding model. (The vector dimensions are different for different embedding models, for example it is 384 for multi-qa-MiniLM-L6-cos-v1 and 768 for nomic-ai/nomic-embed-text-v1). When a question is asked, MyGPT converts the question into a vector using the same embedding model and embeds it into the same vector space as used for the library; then, it finds the nearest 3 to 5 chunks. The number of chunks - k is decided by finding the first 15 chunks that are nearest (i.e., shortest distance in vector space) to the query, sorting the chunks from lowest to highest distance to the query; k is defined as the number where the inter-distance chunk after sorting starts to increase. k is typically 3-5 chunks for most questions in the benchmark datasets (**Supplementary Information**). MyGPT offers the option to utilize the BM25 method for identifying relevant chunks through semantic search, ensuring that no keywords are overlooked. If selected in addition to vector-based contextual search, MyGPT will combine top k results from both the methods. MyGPT will then use the two sets of top k chunks as “context” to generate an answer to the question, effectively prompting the LLM with the context (i.e., relevant retrieved information) to answer the user query.

Running and hosting MyGPT
The MyGPT pipeline can be run on a local computer for personal use or hosted on a server/cloud infrastructure for group, department- or institution-wide usage.

To run MyGPT locally on a personal computer, a user needs a device with at least 8 CPUs, 16 GB RAM, and a GPU. We have provided benchmarking instructions for testing Ollama on personal devices (see GitHub repository for instructions). If users can run Ollama satisfactorily, they can install MyGPT and run the entire pipeline on the same device. Detailed descriptions of the steps for all approaches are available in the GitHub repository (https://github.com/mb-group/MyGPT_public). We have tested installation and LLM performance of MyGPT on various personal devices (details in **Supplementary Table 2**). During our testing, in addition to personal laptops, we were able to install and run MyGPT on an affordable Edge AI compute device – NVIDIA Jetson Orin Nano (cost - $250), with small LLMs such as Llama 3.2 (1b, 3b) and Gemma (2b) (**Extended Data Fig. 6**).

In cases with many MyGPT users, such as departments, institutions, or industry settings, hosting MyGPT on a server or cloud platform would be advantageous. To optimally run the MyGPT pipeline in such environments, the installer needs to understand the operation and maintenance of a web application server. We have tested MyGPT on local servers with various GPUs, including Nvidia GPU GeForce RTX 4090 and Nvidia GPU A100, as well as on cloud infrastructures like Amazon Web Services. However, resource requirements and configurations will vary for each use case (details in **Supplementary Table 1**). Therefore, involving Information Technology personnel to assist in hosting MyGPT for these instances will be helpful.

Question, and Answer Relevance Scores

To calculate Question Relevance Score (QRS), we define QRS as

$$QRS = 1 - \frac{QC_{mean} - QC_{best}}{QC_{worst} - QC_{best}}$$

where QC_{mean} is the mean distance between the embedded question and the k closest embedded chunk in the vector database. In our observations, we have seen that 3 to 5 closest chunks for the question provides an optimal response. QC_{best} and QC_{worst} are empirical highest and lowest QC_{mean} distances that were observed for the library of 232 questions and retrieved context chunks for the benchmark dataset (details in **Supplementary Information**). Both vary for different embedding models. While the default values of QC_{best} and QC_{worst} are based on our public biomedical library benchmark dataset (**Supplementary Information**), users can modify them from customization settings through the UI.

We also defined an Answer Relevance Score (ARS) as

$$ARS = 1 - \frac{AC_{mean} - AC_{best}}{AC_{worst} - AC_{best}}$$

AC_{mean} is the mean distance between the embedded answer (with context) and the k embedded chunks that were used as context (see *identification of chunks for LLM context*). AC_{best} and AC_{worst} are empirical highest and lowest *AC* values observed for the library of 232 questions for the test dataset (details in **Supplementary Information**). While the default values of AC_{best} and AC_{worst} are based on our public biomedical library benchmark dataset (**Supplementary Information**), users can modify them from customization settings through the UI.

We also defined Hallucination Index (HI) by combining QRS and ARS, given by

$$HI = a - (b \times QRS) - (c \times ARS)$$

Where *QRS* is the Question relevance score and *ARS* is the Answer relevance score. The default HI calculation values for *a* is 1, *b* is 0.5 and, *c* is 0.5. These values are selected because of the inverse relationship between the question's relevance and answer's relevance to the context and expected hallucination. If both question and answer are relevant to the documents, the LLM should have used the retrieved context to generate the answer, and hallucination will be minimal. Using the formula, if QRS and ARS are 1, resulting in HI to be zero and vice versa. The user can modify the default values of *a*, *b* and *c* to match the needed level of confidence through the UI.

Benchmarking and usability testing of the MyGPT pipeline

We have tested the MyGPT pipeline for technical and usability aspects. To do this, we asked 10 participants with varying domain knowledge and technical skills to install and test MyGPT on their personal computers. All participants successfully installed MyGPT and used it with their respective libraries. We further benchmarked the performance of MyGPT by running it in various environments, such as on a personal computer, local server with GPU inferencing, and cloud infrastructure. The differences in response times are reported in **Supplementary Table 2**. During our benchmarking test, we noticed that the Gemma model was the fastest in all environments, but the answers were generally shorter and had fewer details than those generated by the larger models such as Llama3:70b. Comparing different environments, the performance of MyGPT on an Apple laptop with M1 GPU (8 CPUs and 64 GB RAM) was equivalent to or better than Google Cloud Platform (GCP) with a T4, L4 or A100 GPU and cloud infrastructure such as Amazon Web Services (AWS) with A10G GPU. For the users who don't have access to GPU via a PC or laptop, or have less powerful laptops, they can either use compact Edge AI devices such as Nvidia Jetson Orin (starts at $250) or use Cloud service with T4 GPU, which can cost around $0.5 per GPU per hour or around $200 per GPU per month (in US market as of September 2024).

Library size benchmarking

We tested different library sizes and their impact on the MyGPT workflow. Instead of focusing on the number of documents, we looked at pages per library, creating four libraries with 10, 100, 1,000, and 10,000 pages. These were tested on two Apple MacBooks with M1 and M4 GPUs (8 CPUs, 64 GB RAM) and a St. Jude Server with A100 GPU (64 CPUs, 2 TB RAM). Uploading and retrieval times were managed by MyGPT backend, while generation time was handled by the LLM server. The end-to-end timing captured the complete Q&A process. Uploading was performed using a fixed-chunking method (1,000-page chunks) with the nomic-embed-text-v1.5 model for embedding. We used llama3:8b for generation on both laptops and the server. Results are reported in **Supplementary Table 4**.

Dataset of questions and answers to evaluate MyGPT on private and confidential document

To verify the usability of MyGPT for private documents, we used the MyGPT manuscript as a private document and performed a manual evaluation since this manuscript would not have been included in the LLM training set. For that, we curated a dataset of 40 questions related to the MyGPT manuscript from 2 domain experts. Out of those 40 questions, 25 were related to information present in the manuscript (relevant questions), and 15 were related to the manuscript's topic but lacked information in the manuscript to answer the question (irrelevant questions). This is provided in the **Supplementary Information**. Domain experts provided at least five relevant questions per question type (n=25) and 1-5 irrelevant questions per question type (n=15).

Manual evaluation of MyGPT on private, confidential documents and performance evaluation

The expert-generated questions for private documents were answered by MyGPT using Llama2 and Llama2:7b (alone, without context). We used the same customization settings as for the biomedical library (all-MiniLM-L6-v2 embedding model, a chunk size of 1000 characters without overlaps, the squared Euclidean distance metric, and Ollama's default setting for temperate (0.8), top-P (0.9) and top-K (40)) to generate the answers. Experts were asked to provide feedback on whether the individual answers generated from MyGPT and Llama2 were correct or incorrect.

MyGPT libraries for the different use cases
To demonstrate the application of MyGPT for research, clinical task and global health policy at St. Jude Research Hospital, we created multiple instances of MyGPT.

- For literature review of biomedical articles, we created a library of articles on G protein-coupled receptors (GPCRs). The library was created using the nomic-embed-text:latest embedding model, with structure-preserving chunking and an overlap of 20% between the chunks. The Squared L2 distance function and the BM25 algorithm were used for keyword matching. We utilized gpt-oss:20b LLM to generate an answer.
- For the clinical document, we created a MyGPT library with Standard Operating Procedures (SOPs) from the Department of Bone Marrow Transplantation and Cellular Therapy (BMTCT) at St. Jude. The library was created using the nomic-embed-text:latest embedding model, with a fixed chunking method of 1000 characters, an overlap of 20% between chunks, a Squared L2 distance function, and the BM25 algorithm for keyword matching. We utilized llama3.1:latest LLM to generate an answer.
- For Q&A with multi-lingual documents pertaining to global health, we created a library containing health policies, with records available in multiple languages (English, Spanish, and French) from the St. Jude Global Pediatric Medicine Department. The library was created using the bge-m3:latest embedding model, with a fixed chunking method of 1000 characters and an overlap of 20% between the chunks, utilizing the Squared L2 distance function. We utilized llama3.1:latest LLM to generate an answer.

Acknowledgments

We acknowledge ALSAC for supporting this work. We also acknowledge support from the Blue-Sky Kinase initiative (J.P., J.D., D.M., B.M. and M.M.B) and GPCR Collaborative (J.D., V.T., and M.M.B) funding mechanisms within St Jude. We thank the domain experts – Drs. Balaji Santhanam, Katarina Nemec, Preethi Badrinarayan, Katalin Parej, Ben Leslie, Kalyan Immadisetty, Benjamin Lang for helping create the Q&A datasets used for benchmarking the performance of MyGPT. We thank Drs. Jim Downing, Paul Taylor, Babis Kalodimos (Structural Biology), Carlos Rodriguez-Galindo (Global Pediatric Medicine), Stephen Gottschalk (BMTCT), Elly Park and the Blue-Sky Kinase team for comments on this work as well as Ashok Kumar Reddy Boddu, Ed Suh, Michael Brewer, Darrell Gentry, Chad Burdyshaw, and Keith Perry for their support from Information Services department within St. Jude.

Author Contributions

J.P. and M.M.B. conceived the MyGPT concept and all confidence metrics. J.P. and J.D. contributed equally to designing a prototype for MyGPT software. J.P. developed the MyGPT software and pipeline. B.M., V.T., and D.M. helped with multiple rounds of manual evaluation of the benchmark datasets. D.M. reviewed the code. H.F. implemented the recursive chunking approach. S.D.M.A assisted with analyzing confidence metrics values for the benchmark dataset. S.A. installed and supported Ollama on a server with A100 GPU, D.A. set automation pipeline and GitHub repository management, A.W. implemented frontend unit tests and A.W. and D.A. set up cloud Azure VMs. J.P., M.S., I.C., and M.M.B. wrote the manuscript. J.P. led the project. M.M.B. managed and set the overall vision and specific directions of the project.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information is available for this paper.

Correspondence and request for materials should be addressed to Jaimin Patel or M. Madan Babu.

Code availability

All code, data, and tools are available at https://github.com/mb-group/MyGPT. [This link is currently private and will be made public upon acceptance for publication in a journal. We have provided the source code as zip file with supplementary information for the review process. We have also provided a URL with username and password for each reviewer to the journal editor. Please contact the journal editor if you prefer to evaluate the hosted version of MyGPT]. We have performed vulnerability testing using Snyk to evaluate best practices in software development to eliminate security weaknesses. Our code and data are licensed under GPL-3.0 license, while LLM models and LLM tools used for the project are available under the licenses of the respective sources.

EXTENDED DATA TABLES

Extended Data Table 1: Example LLMs that can be used with MyGPT via Ollama.

LLM	Parameters	Size	Developed by	Note	Ideal hosting environment	Minimum requirements
Llama2:7b	6.7B	3.8 GB	Meta	Released on 18th July 2023	PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Llama3:8b	8.03B	4.7 GB			PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Llama3:70b	70.6B	40 GB			VM, server, Cloud service	1 GPU, 64 GB RAM
Llama3.1:8b	8.03B	4.9 GB		Released on 23rd July, 2024	PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Llama3.2:1b	1.24B	1.3 GB		Released on 25th September, 2024	Edge AI device, PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Llama3.3:70b	70.6B	43 GB		Released on 7th December, 2024	VM, server, Cloud service	1 GPU, 64 GB RAM
Gemma:7b	8.54B	5 GB	Google DeepMind	Released on 21st February 2024	PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Gemma2:9b	9.24B	5.4 GB		Released on 27th June 2024	PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Gemma2:2b	2.61B	1.6 GB			Edge AI device, PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Gemma3:1b, 4b	1B, 4.3B	0.8GB, 3.3GB		Released on 12th March 2025	Edge AI device, PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Gemma3:12b, 27b	12.2B, 27.4B	8.1GB, 17GB		Released on 12th March 2025	PC/Laptop, Cloud service	1 GPU, 32 GB RAM
Gpt-oss:20b	20.9B	14GB	OpenAI	Released on 5th August 2025 (Thinking model)	PC/Laptop, Cloud service	1 GPU, 32 GB RAM
DeepSeek-R1:8b, 14b	8.19B, 14.8B	5.2GB, 9GB	DeepSeek	Released on January 20, 2025 (Thinking model)	PC/Laptop, Cloud service	1 GPU, 32 GB RAM
Qwen3:8b, 14b	8.19B, 14.8B	5.2GB, 9.3GB	Alibaba	Released on April 29th, 2025 (Thinking model)	PC/Laptop, Cloud service	1 GPU, 16 GB RAM
Mistral:7b	7.2B	4.1 GB	Mistral AI	Released on 27th September 2023	PC/Laptop, Cloud service	1 GPU, 8 GB RAM
Vicuna:7b	6.74B	3.8 GB	LMSYS Org, UC Berkeley	Finetuned Llama2	PC/Laptop, Cloud service	1 GPU, 8 GB RAM

Extended Data Table 2: LLM performance for different environments.

LLM size category	LLM	Host environment	Infrastructure	GPU	Avg. Speed (seconds)
Mid-size LLMs (most common)	**Llama2:7b**	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	3.80
		Google Cloud Platform	2 vCPUs, 12 GB RAM	1 Nvidia T4 Tensor Core	6.46
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	2.89
		Apple Mac m1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	9.26
		Apple Mac m3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	5.21
	Llama3:8b	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	3.34
		Google Cloud Platform	2 vCPUs, 12 GB RAM	1 Nvidia T4 Tensor Core	4.96
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	3.40
		Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	7.47
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	6.00
	Llama3.1:8b	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	2.49
		Google Cloud Platform	2 vCPUs, 12 GB RAM	1 Nvidia T4 Tensor Core	6.31
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	2.05
		Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	7.05
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	4.45
	Gemma:7b	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	2.74
		Google Cloud Platform	2 vCPUs, 12 GB RAM	1 Nvidia T4 Tensor Core	4.38
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	1.61
		Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	5.52
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	2.80
	Gemma2:9b	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	2.72
		Google Cloud Platform	2 vCPUs, 12 GB RAM	1 Nvidia T4 Tensor Core	4.57
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	1.46
		Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	6.01
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	3.28
	Mistral:7b	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	3.37
		Google Cloud Platform	2 vCPUs, 12 GB RAM	1 Nvidia T4 Tensor Core	5.69
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	2.67

		Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	7.87
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	5.28
	Vicuna:7b	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	3.96
		Google Cloud Platform	2 vCPUs, 12 GB RAM	1 Nvidia T4 Tensor Core	8.73
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	3.56
		Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	9.51
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	8.93
Small LLMs	**Llama3.2:3b**	Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	3.79
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	2.28
		Nvidia Orin-nano	1 CPU, 8 GB RAM	1 1024-core NVIDIA Ampere	19.88
	Llama3.2:1b	Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	2.15
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	1.52
		Nvidia Orin-nano	1 CPU, 8 GB RAM	1 1024-core NVIDIA Ampere	9.82
	Gemma:2b	Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	1.25
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	0.80
		Nvidia Orin-nano	1 CPU, 8 GB RAM	1 1024-core NVIDIA Ampere	5.26
Large LLMs	**Llama3:70b**	Google Cloud Platform	2 vCPUs, 83 GB RAM	1 Nvidia A100 Tensor Core	11.17
		St. Jude internal server	64 CPUs, 2 TB RAM	1 Nvidia A100 Tensor Core	31.11
		Apple Mac M1	8 CPUs, 64 GB RAM	1 Apple M1 Metal	60.27
		Apple Mac M3	8 CPUs, 64 GB RAM	1 Apple M3 Metal	49.53

Extended Data Table 3. Cost of hosting LLMs on cloud infrastructure.

Cloud service provider	Infrastructure	GPU	Avg. cost per month (USD)
Azure	24 vCPUs, 220 GB RAM	1 Nvidia A100 Tensor Core	$2,628.00
Azure	8 vCPUs, 56 GB RAM	1 Nvidia T4 Tensor Core	$538.01
Google	12 vCPUs, 85 GB RAM	1 Nvidia A100 Tensor Core	$2,682.57
Google	8 vCPUs, 30 GB RAM	1 Nvidia T4 Tensor Core	$374.03
Google	4 vCPUs, 16 GB RAM	1 Nvidia L4 Tensor Core	$516.99

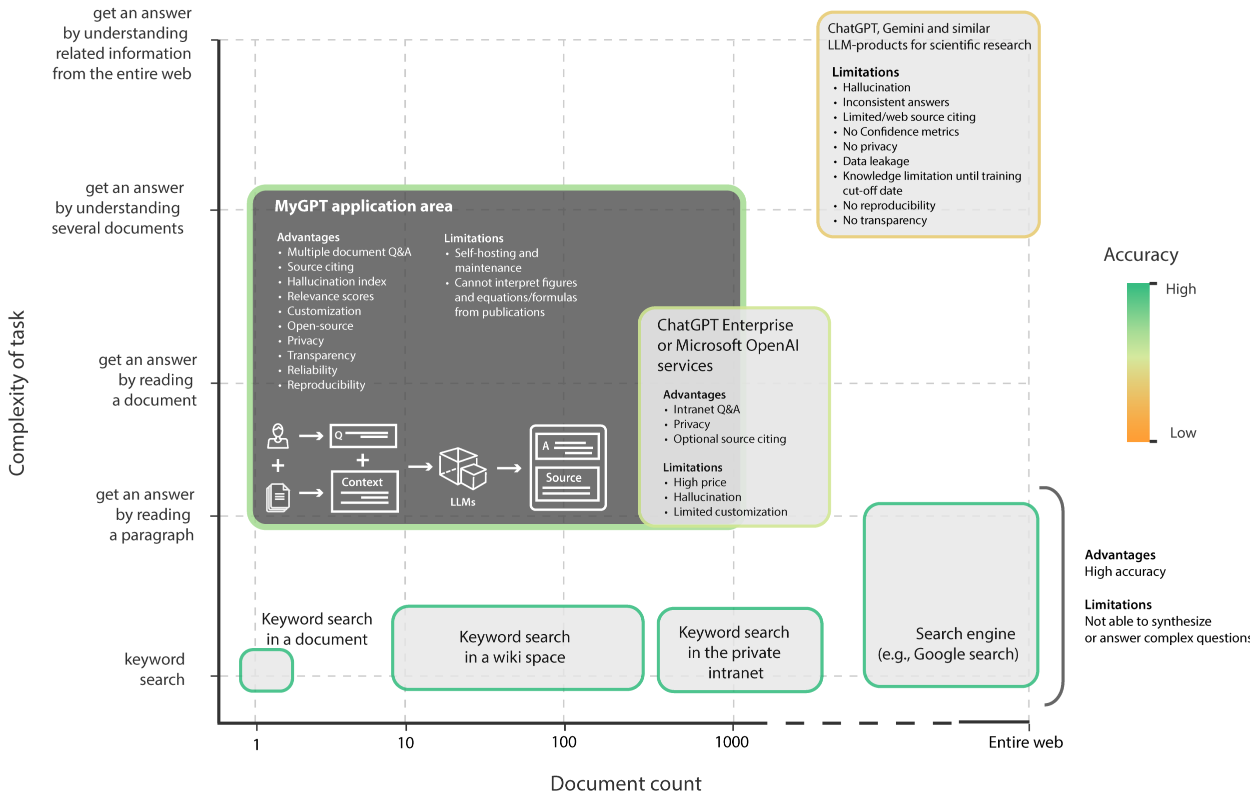
Extended Data Table 4. Benchmarking of various MyGPT steps for different library sizes.

Device	Pages	PDF count	Chunks count (fixed chunk size)	Upload time	Retrieval time (1st)	Retrieval time (avg)	Generation time (1st) llama3:7b	Generation time (avg)	end-to-end time (avg)
Mac Laptop (M1 chip)	10	1	70	2m10s	1.28s	1.06s	6.39s	2.99s	11s
	100	2	235	2m33s	1.18s	0.62s	4.16s	6.03s	17s
	1000	4	3211	5m53s	0.73s	0.59s	3.16s	3.9s	12s
	10000	791	58869	95m19s	1.94s	0.64s	5.52s	5.82s	17s
Mac Laptop (M4 chip)	10	1	70	2.05s	0.38s	0.225s	0.84s	1.205s	4.5s
	100	2	235	4.9s	0.1s	0.0775s	2.04s	2.295s	4.5s
	1000	4	3211	1m18s	0.14s	0.11333s	1.9s	2.049s	4.2s
	10000	791	58869	23m51s	0.69s	0.651s	1.86s	2.124s	5.0s
SJ VM	10	1	70	11s	5.63s	0.80s	11.22s	0.51s	11s
	100	2	235	13s	0.77s	0.79s	1.2s	1.07s	6.75s
	1000	4	3211	2m12s	0.68s	0.73s	1.16s	1.49s	12.6s
	10000	791	58869	46m2s	0.77s	0.85s	1.19s	1.39s	8s

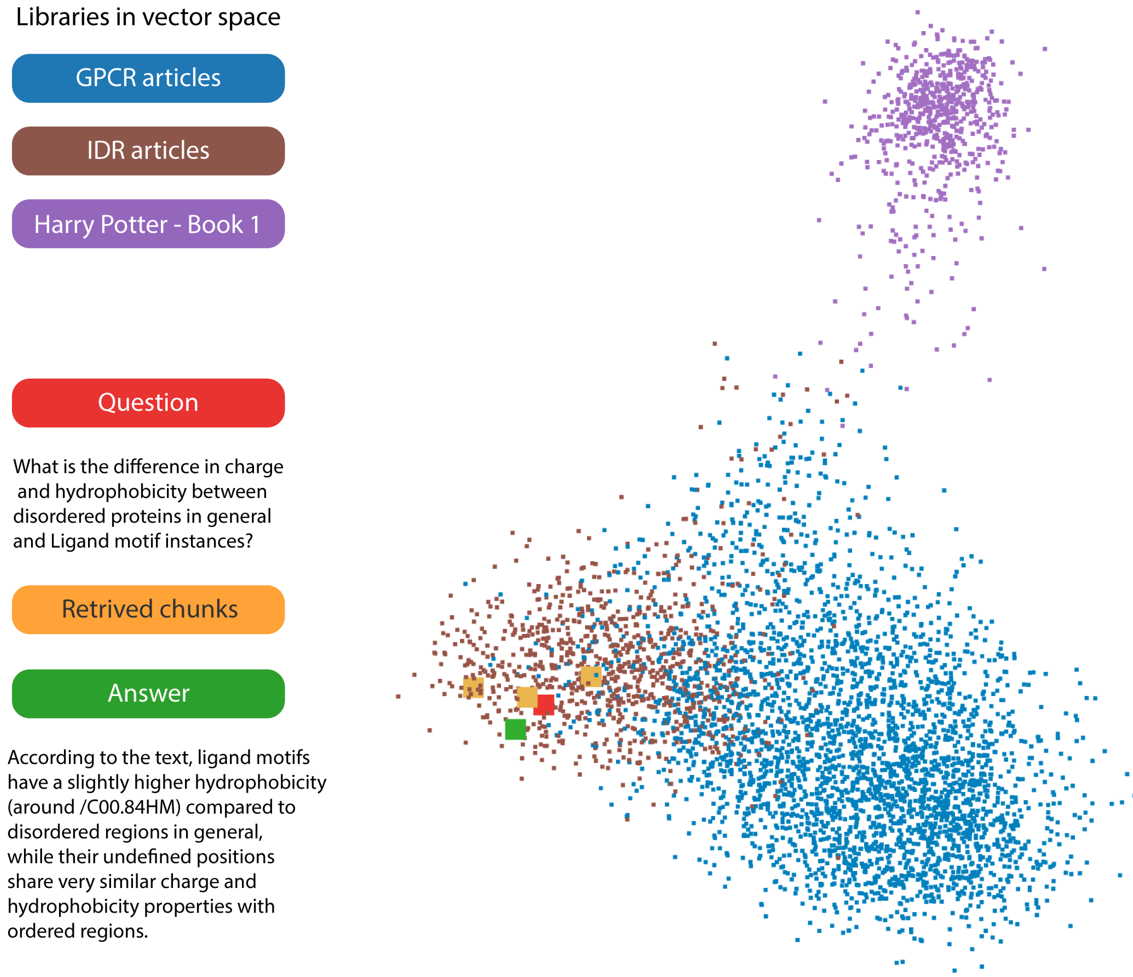
Extended Data Table 5. Comparison of various commercial and open-source RAG solutions.

Solution	Business model	Software type	Note
MyGPT	Open-source	end-to-end	Fully open-source pipeline with relevance scores, hallucination detection with PDF highlighting and user and library management
ChatPDF	Commercial	end-to-end	Commercial product powered by OpenAI models
NotebookLM	Commercial	end-to-end	Commercial product by Google
PrivateGPT	Open-source	end-to-end	Free demo software with limited features; enterprise, deployable and full product managed by Zylon AI
Verba	Open-source	Front-end	Demo RAG app to work with commercial vector database developed by Weaviate
LlamaIndex	Semi open-source	Package	Semi open-source library to create LLM-powered applications, premium features behind paywall
BioChatter	Open-source	Separate Front-end and back-end	LLM based app to chat with BioCypher knowledge graphs, basic RAG capability

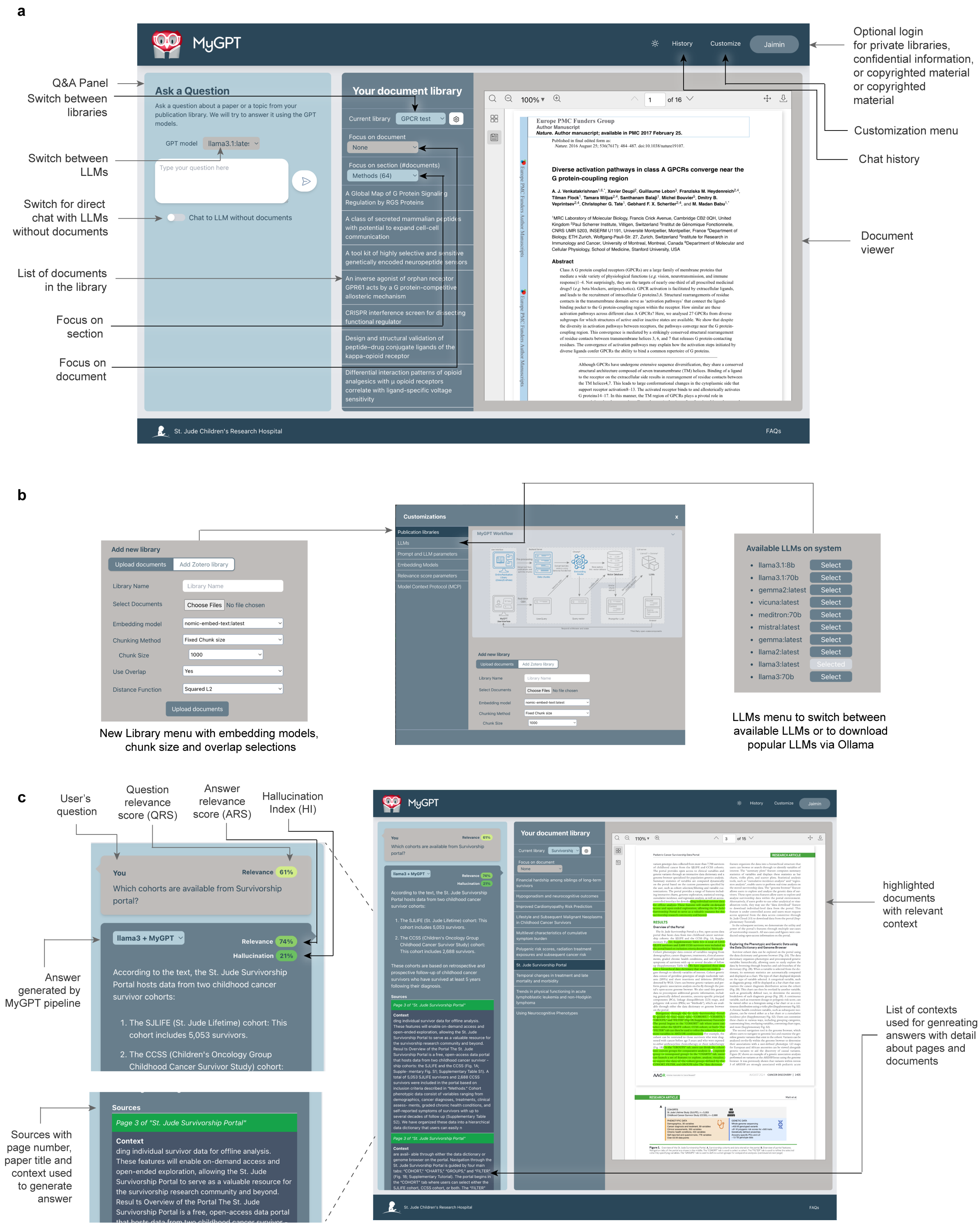
EXTENDED DATA FIGURES



Extended Data Figure 1. **Information-seeking tasks and solutions**. Keyword searches from a document, several documents, an intranet, or the entire internet are some of the routine tasks for most knowledge workers, and these have been optimized over the past several decades to provide accurate information. With ChatGPT and similar products, users can get answers to complex questions requiring processing of related material, but accuracy and factuality are lower than in typical search solutions. MyGPT implements the RAG concept and fills a void in the landscape of available discovery tools, highlighting an unmet and previously unidentified need to get deeper insights from related documents. The color of the boxes indicates the accuracy of the information obtained from each task, as represented in the scale bar on the right.

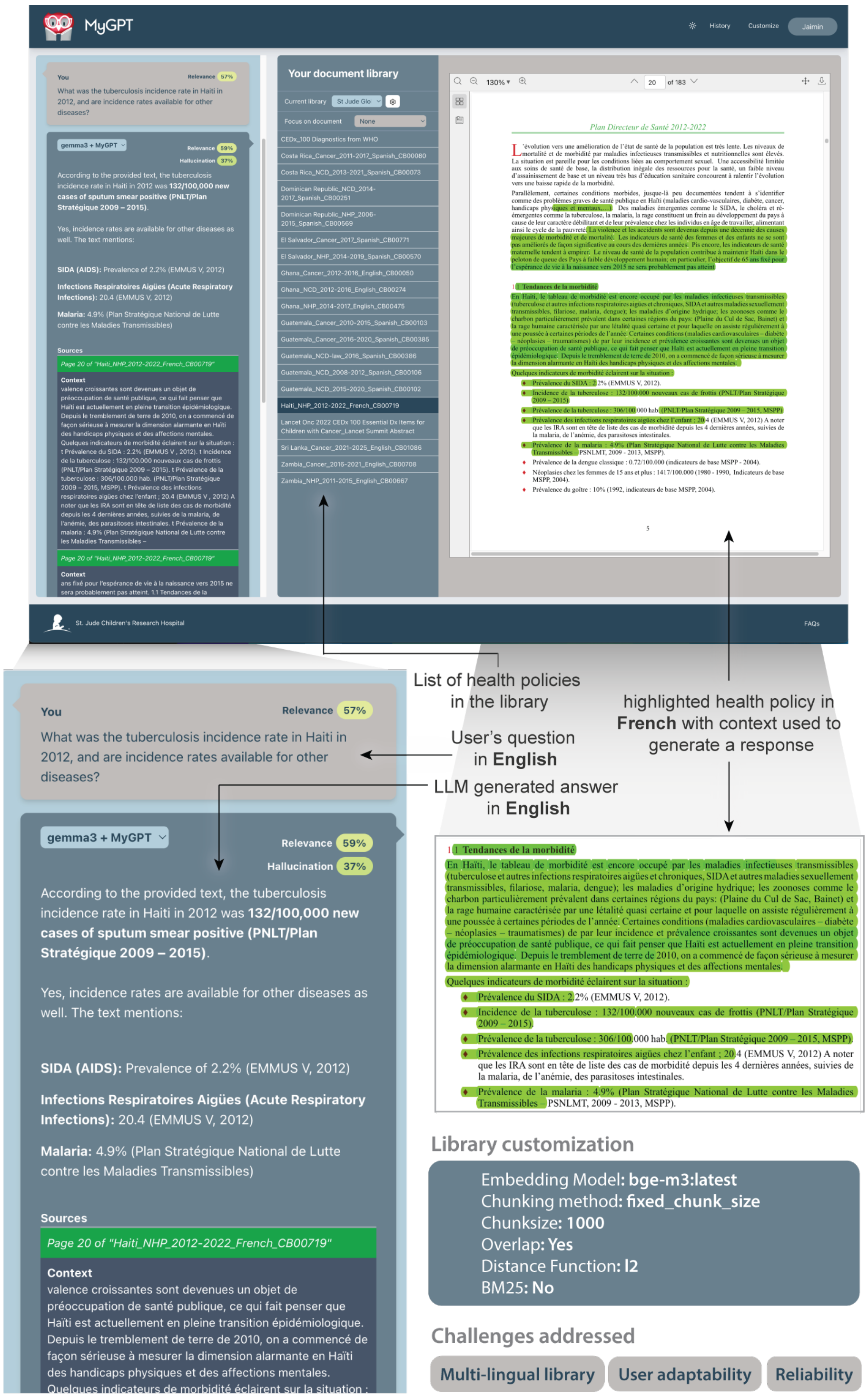


Extended Data Figure 2. Visualization of chunks from documents in the library in the vector space of embeddings. The PCA plot visualizes vector embeddings of chunks from two different libraries of publicly available biomedical articles on (a) G protein-coupled receptors (GPCRs; 27 articles) in blue, (b) intrinsically disordered regions (IDRs; 8 articles) in brown, and the text from (c) Harry Potter (Book 1) in purple (Benchmark dataset; **Supplementary Information**). Each dot in the plot denotes a chunk from the three different libraries. A question related to the IDR library is converted into a vector (red square). MyGPT then identifies the chunks (orange squares) that are closest to the question and provides them as context for the LLM to generate a response. The generated answer (green square), when converted into a vector using the same embedding model, is closer to the context, highlighting similarity.



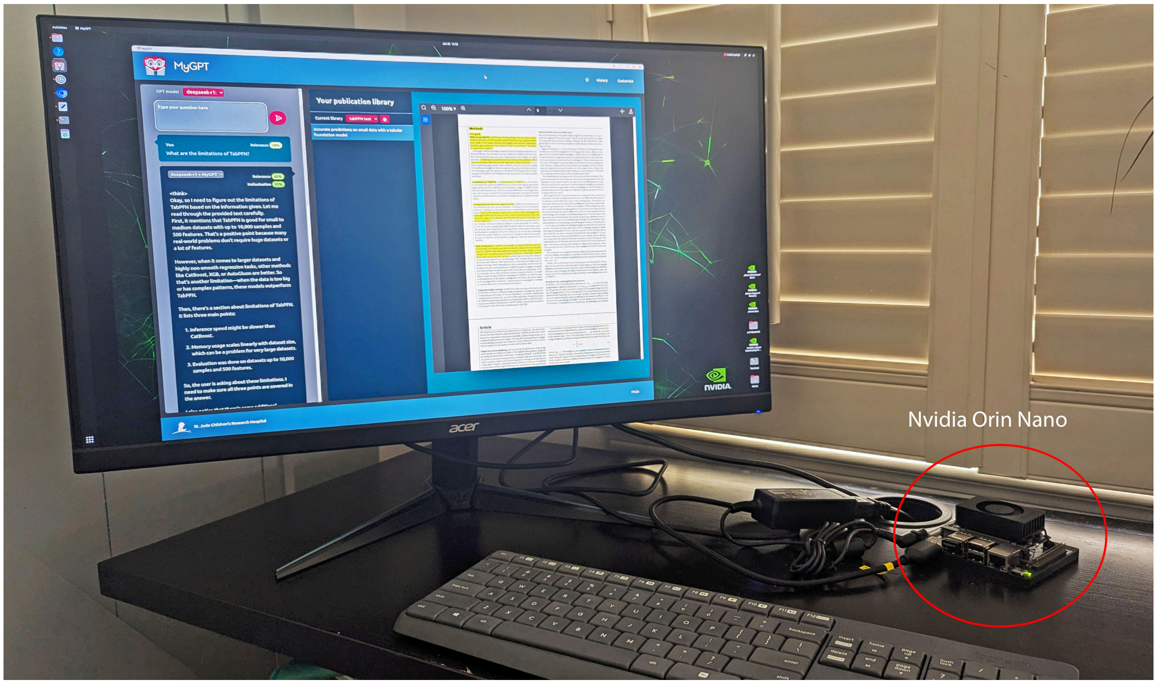
Extended Data Figure 3. User interface for MyGPT. a, MyGPT UI home screen. The home screen shows a chat box on the left panel with the option to switch between different pre-loaded document libraries and different LLMs and a toggle to allow users to chat directly with the LLM without interacting with the documents. In the middle, is a document panel with a list of documents found within the library, and the first document is loaded and visible in the document viewer on the right panel. The top panel, right side shows the icons to enable user login for private libraries (e.g. group or department level access). Users can also check chat history for various libraries and access the settings menu from this area. **b,** MyGPT customization settings. The settings menu provides easy access to MyGPT customizations. Users can upload different libraries under the document library section. Each preprocessed library will be accessible from the MyGPT home page. When uploading, users will also select an embedding model, chunk size, and overlap options. Users can also add new LLMs from the LLMs menu or change the default prompt and temperate, Top-K and Top-P, to adjust creativity vs precision settings from within the settings menu. **c,** MyGPT UI for real-time Q&A. When the library is ready, users can ask questions from the chat box on the home page. MyGPT will first calculate a question relevance score (QRS) and display the value next to the question while it's generating the answer. Next, it will retrieve the relevant context from the documents and provide it to LLM. While it's generating the response, MyGPT will display the chunks with exact pages and paragraphs from the document from where it retrieved the information. Finally, when the answer is generated, it will calculate the answer relevance score (ARS), and hallucination index (HI) and will display them next to the generated response.

Extended Data Figure 4. Use case: Standard Operating Procedures (SOPs) for Bone Marrow Transplantation. Example of a question and answer for the library of private SOPs for the Bone Marrow Transplantation Library at St. Jude. The example shows the user's question and answer, along with the highlighted section of the SOP used by LLM (Llama3.1:latest) to generate the answer. This use case addresses critical challenges in a focused clinical environment where reliability and transparency are the highest priorities. The library was created using the nomic-embed-text:latest embedding model, which incorporates BM25 for precise keyword search, in addition to context-based semantic search via a vector database.



Extended Data Figure 5. **Use case: Global health policies in multiple languages.** Example of a question and answer for the library of global health policies, with documents in multiple languages (English, Spanish, and French) for the St. Jude Global Health Policy Library. The example illustrates a user's question posed in plain English, which was then compared across various sections of the documents, regardless of language using an embedding model with multi-lingual support(bge-m3:latest).

The context in French was utilized by the language model (Llama3.1:latest) to generate the answer in plain English. This use case addresses the challenges of language barriers in global collaborations, where the need for interpreters or experts in multiple languages can hinder productivity.



Extended Data Figure 6. MyGPT running on Nvidia Orin nano with small size LLMs ideal for edge-devices. These include LLMs such as Llama3.2:3b, Llama3.2:1b, Gemma2:2b, DeepSeek-R1:1.5b.